05: Bus Interfaces

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

- P1 due Friday (tomorrow)
- P2 is live

  ↳ multi Processing

Optimizations thus far

- Algorithmic complexity ✓
- Removing redundant computation ✓
- Multithreading ✓
- Multiprocessing* ✓
- Python/C Interfacing ~ 15-20x range

"Bomb" Lab

201 → x86 Assembly

x86 → built in Popcount "POPCNT"

Did you know you can call assembly from C?

```
int popcount_asm(uint64_t num)
{
    uint64_t result;
    asm (
        "POPCNT %1, %0    \n"
        : "=r" (result)
        : "mr" (num)
        : "cc"
    );
    return result;
}
```

gcc bit

```
gcc -Wall -march=nehalem -o test.o test.c popcount.c
```

Optimizations thus far

- ✓ Algorithmic complexity
- ✓ Removing redundant computation
- ✓ Multithreading
- Multiprocessing*
- ✓ Python/C/Asm Interfacing

Can we improve performance with Python->Verilog interfacing?

Oh Yes!

We could also map popcount to hardware

```
import cPopcount
print (cPopcount.cPopcount(0))

import hwPopcount
print hwPopcount.hwPopcount(0))
```

First we need to understand how CPUs and hardware communicate.

We're going on an excursion into bus~~s~~es.

Buses: The hardware interface

$I^2C \Rightarrow$ Inter-Integrated Circuit Bus

SPI $\rightarrow$ Serial Peripheral Interface

- Standardized hardware interfaces for transferring data

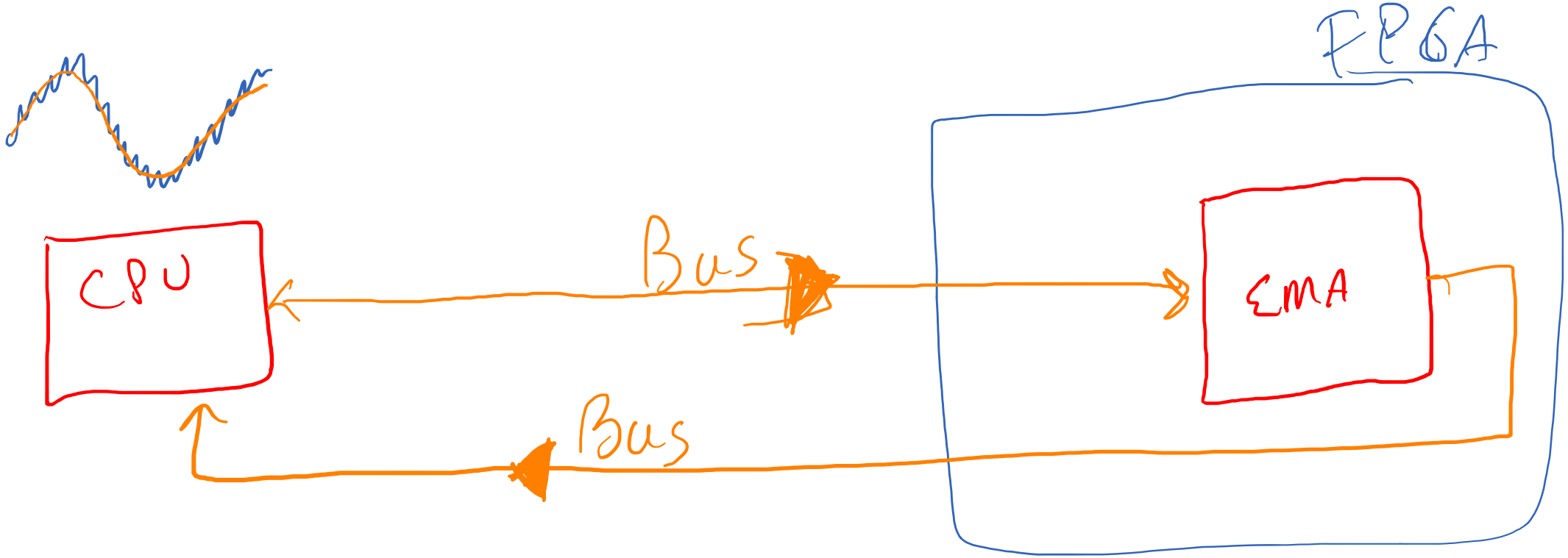
I^2C

- Off-chip Buses: UART, I2C, SPI, RS-485, CAN, Ethernet
- On-Chip Buses: Wishbone, AHB/APB, AXI

- We're going to study 2.
  - AXI4-Stream
  - AXI4-Lite

Advanced
eXtensensible
Interface

P3 “EMA” uses a bus to move data between CPU + hardware



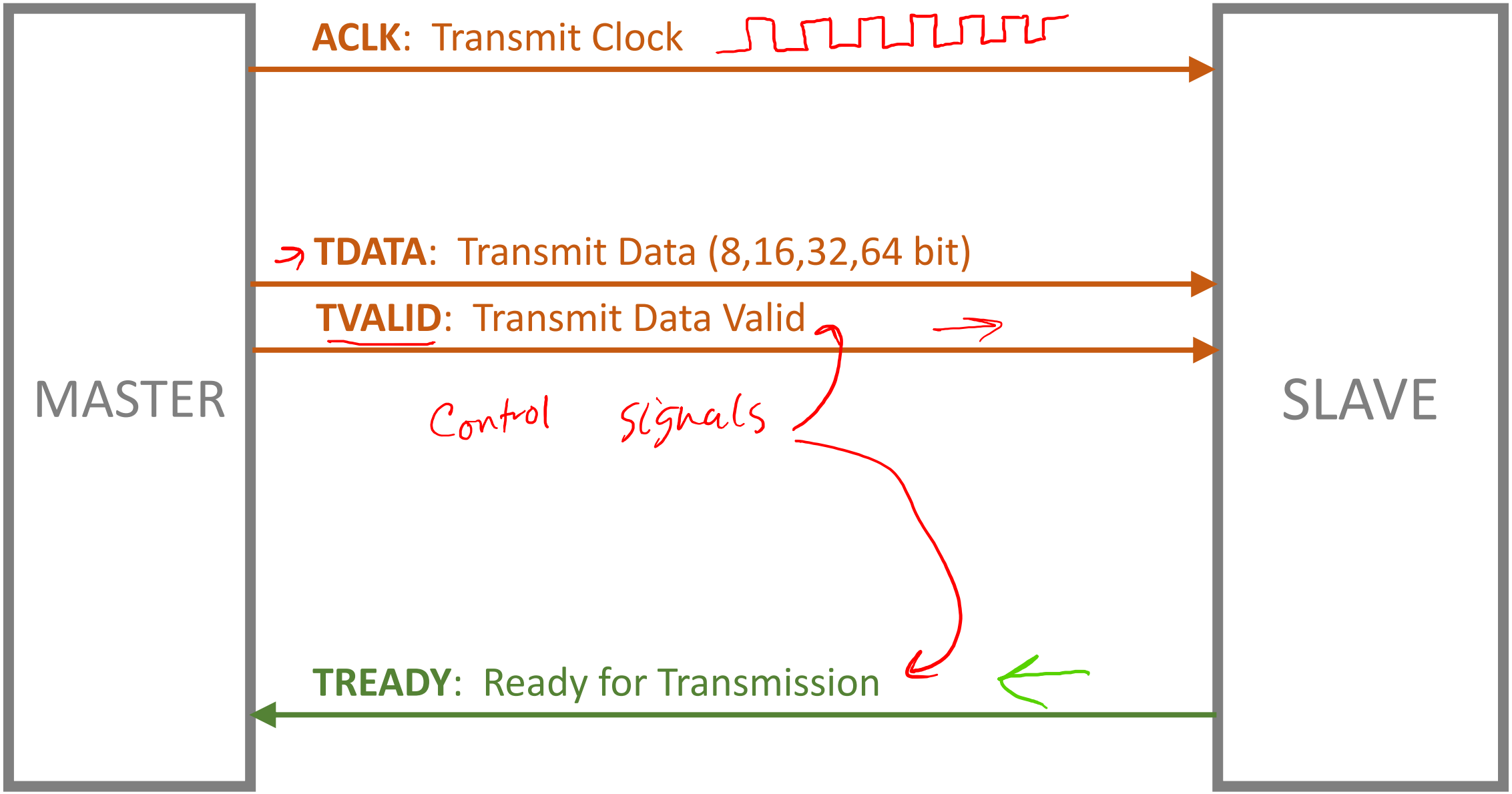
Bus terminology

exchange of data (1 piece of data multiple pieces of data)

- A "transaction" occurs between an "*initiator*" and "*target*"
- Any device capable of being an initiator is said to be a "*bus master*"
  - In many cases there is only one bus master (*single master* vs. *multi-master*).
- A device that can only be a target is said to be a "slave device".

master

slave


MASTER
ACLK: Transmit Clock
TDATA: Transmit Data (8,16,32,64 bit)
TVALID: Transmit Data Valid
Control signals
TREADY: Ready for Transmission
SLAVE

on TDATA

Data is only transferred when

TVALID is **1.**

This indicates the **MASTER** is trying to transmit new data.

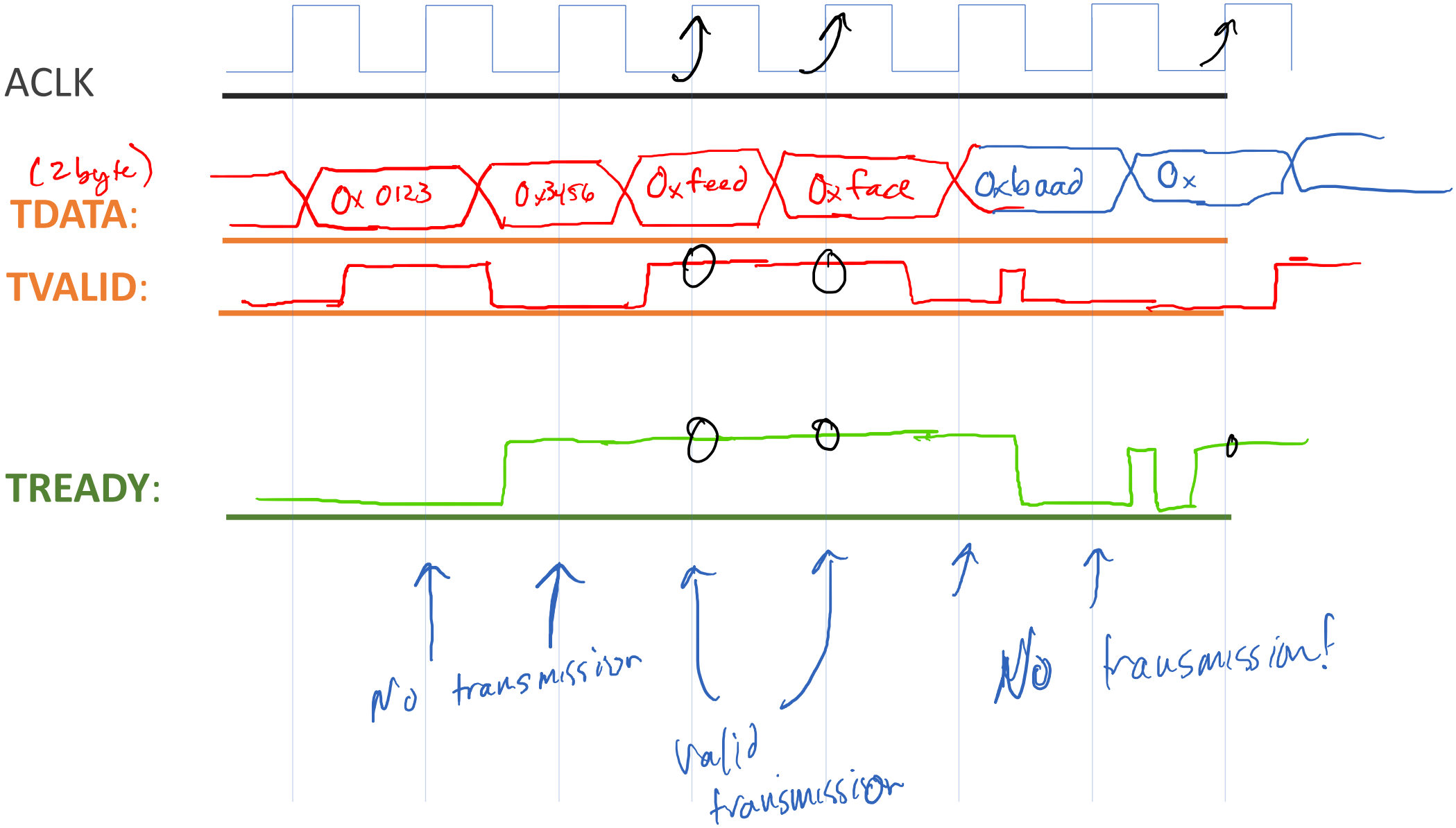
TREADY is **1.**

This indicates the **SLAVE** is ready to receive the data.

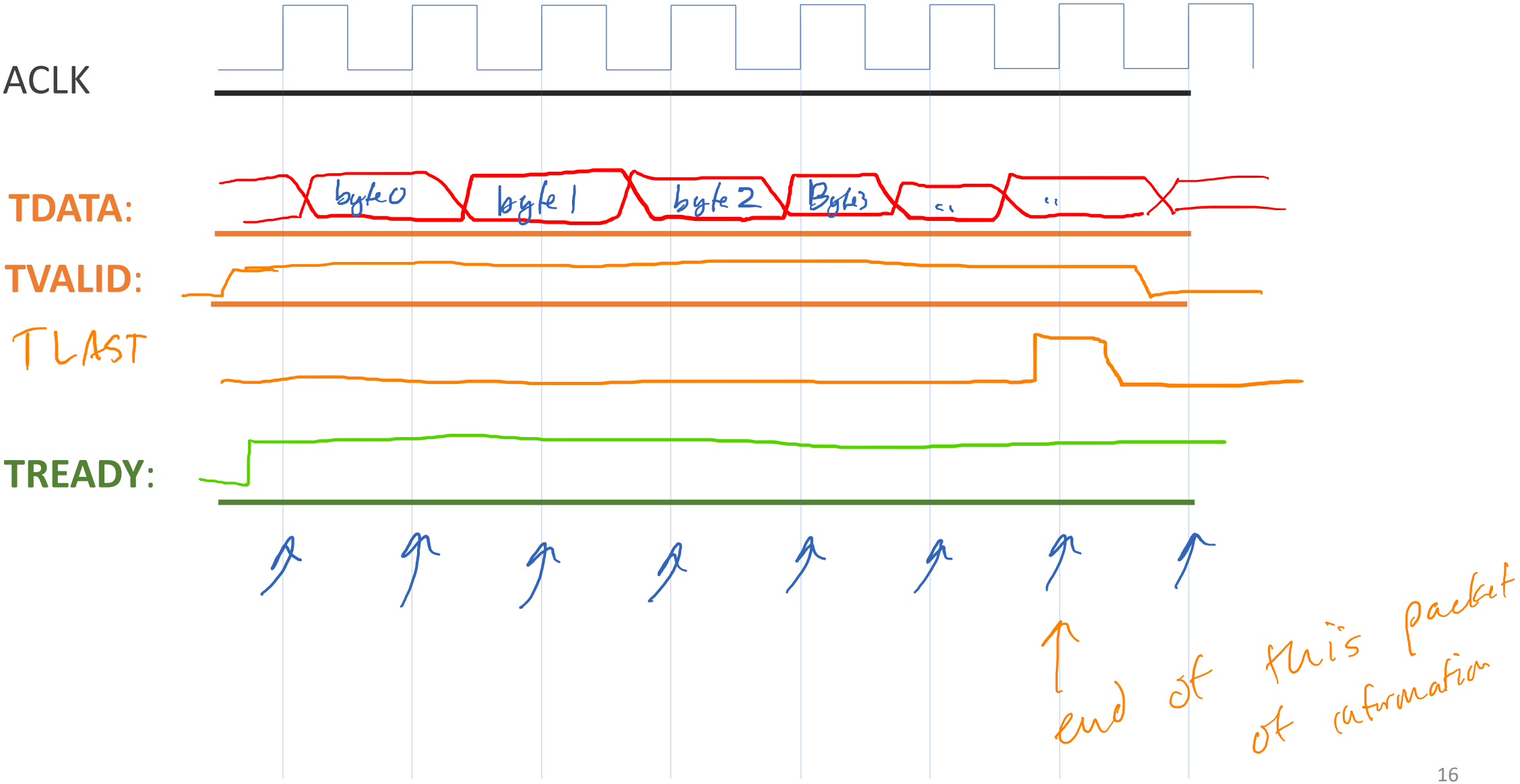
@ positive edge of clock

If either **TVALID** or **TREADY** are 0, no data is transmitted.

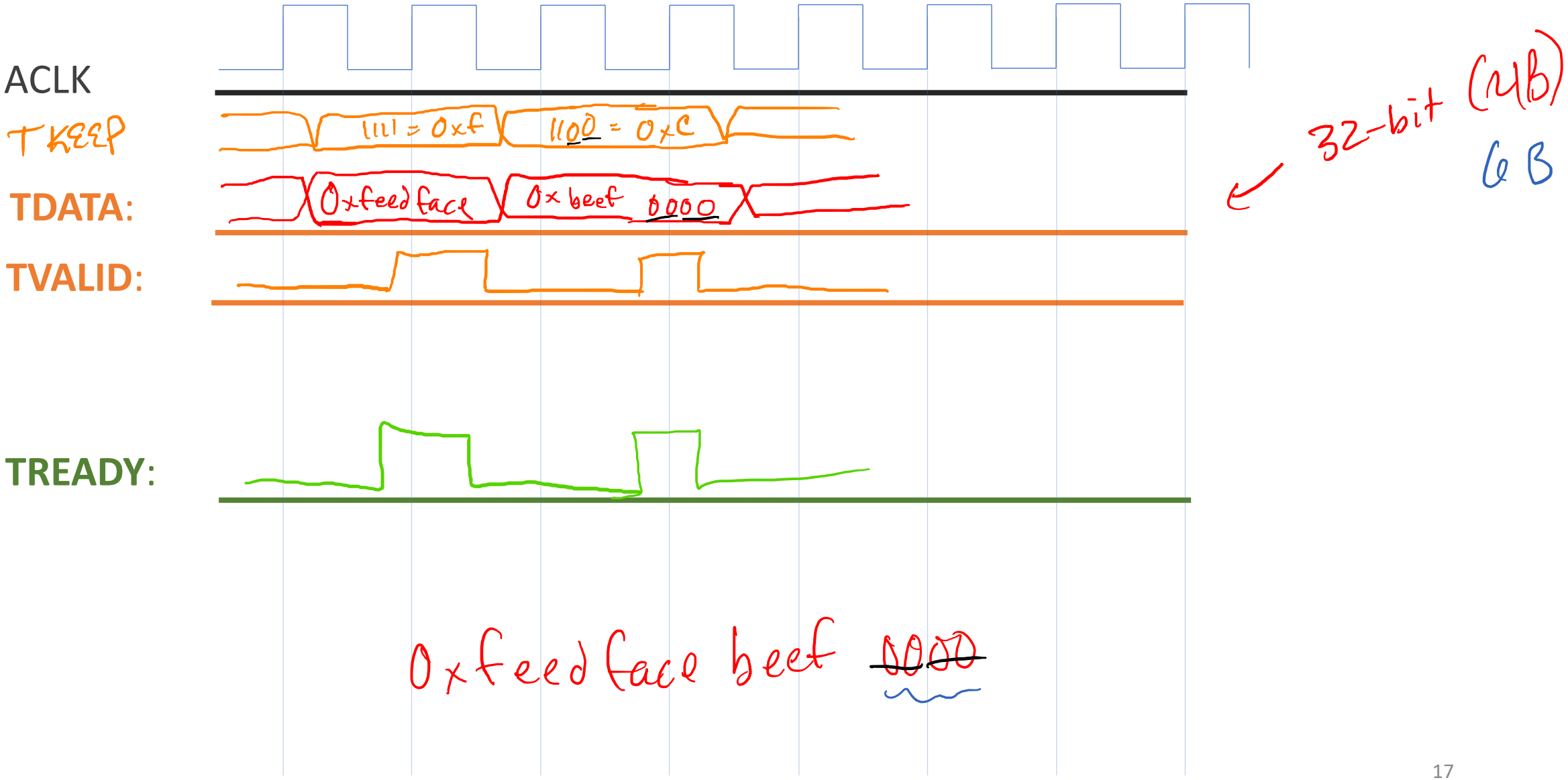
Transferring data on a AXI4-Stream Bus.

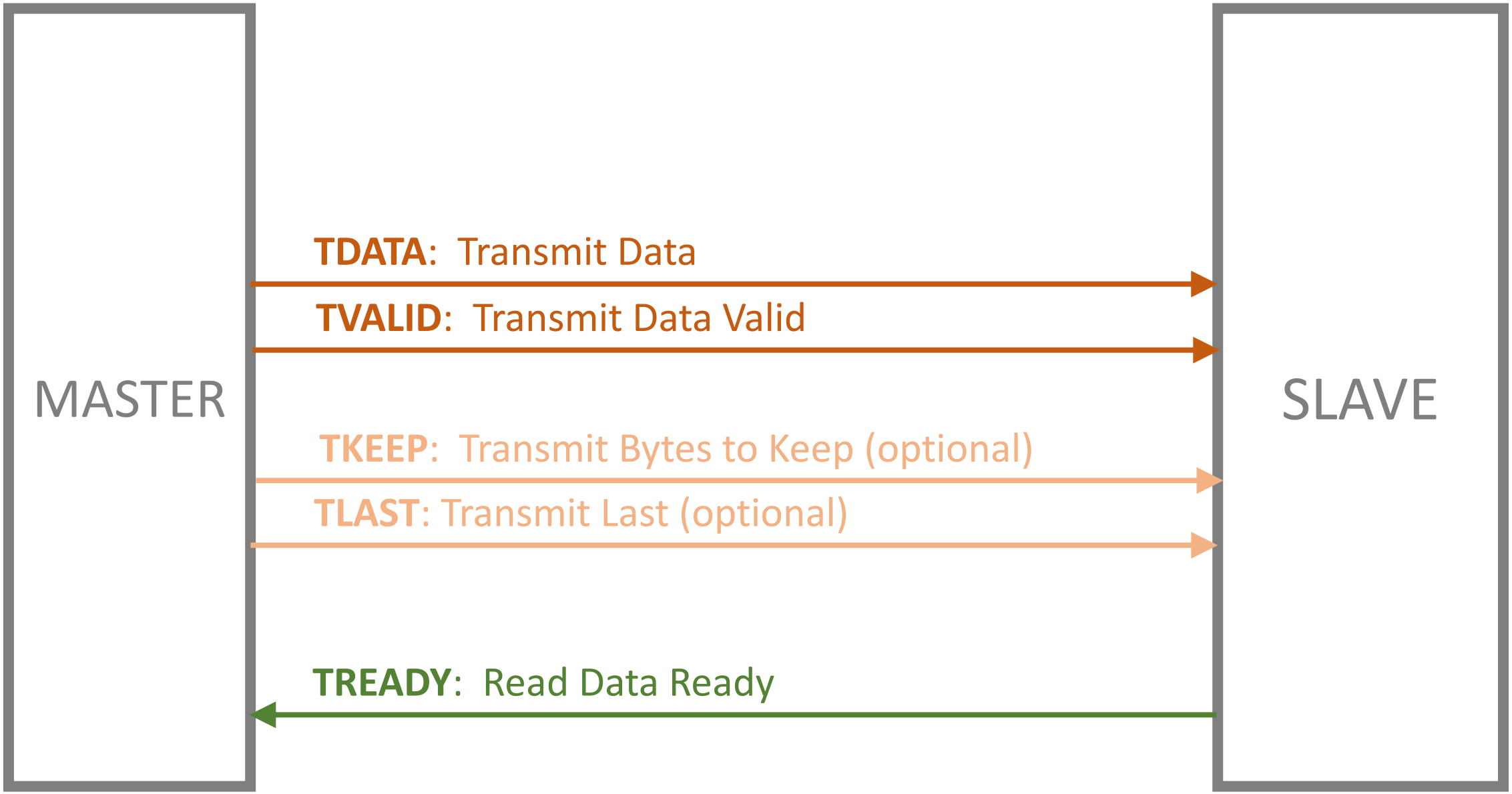


Transferring data on a AXI4-Stream Bus.



Transferring data on a AXI4-Stream Bus.




MASTER
SLAVE
TDATA: Transmit Data
TVALID: Transmit Data Valid
TKEEP: Transmit Bytes to Keep (optional)
TLAST: Transmit Last (optional)
TREADY: Read Data Ready

TKEEP

- What if TDATA is 32-bits (4 bytes) wide, and I want to send 6 bytes?

- **TKEEP** let's me specify which bytes to "keep".

0xF =
- **TKEEP** ==1111 -> Keep all 4 bytes (32-bits)

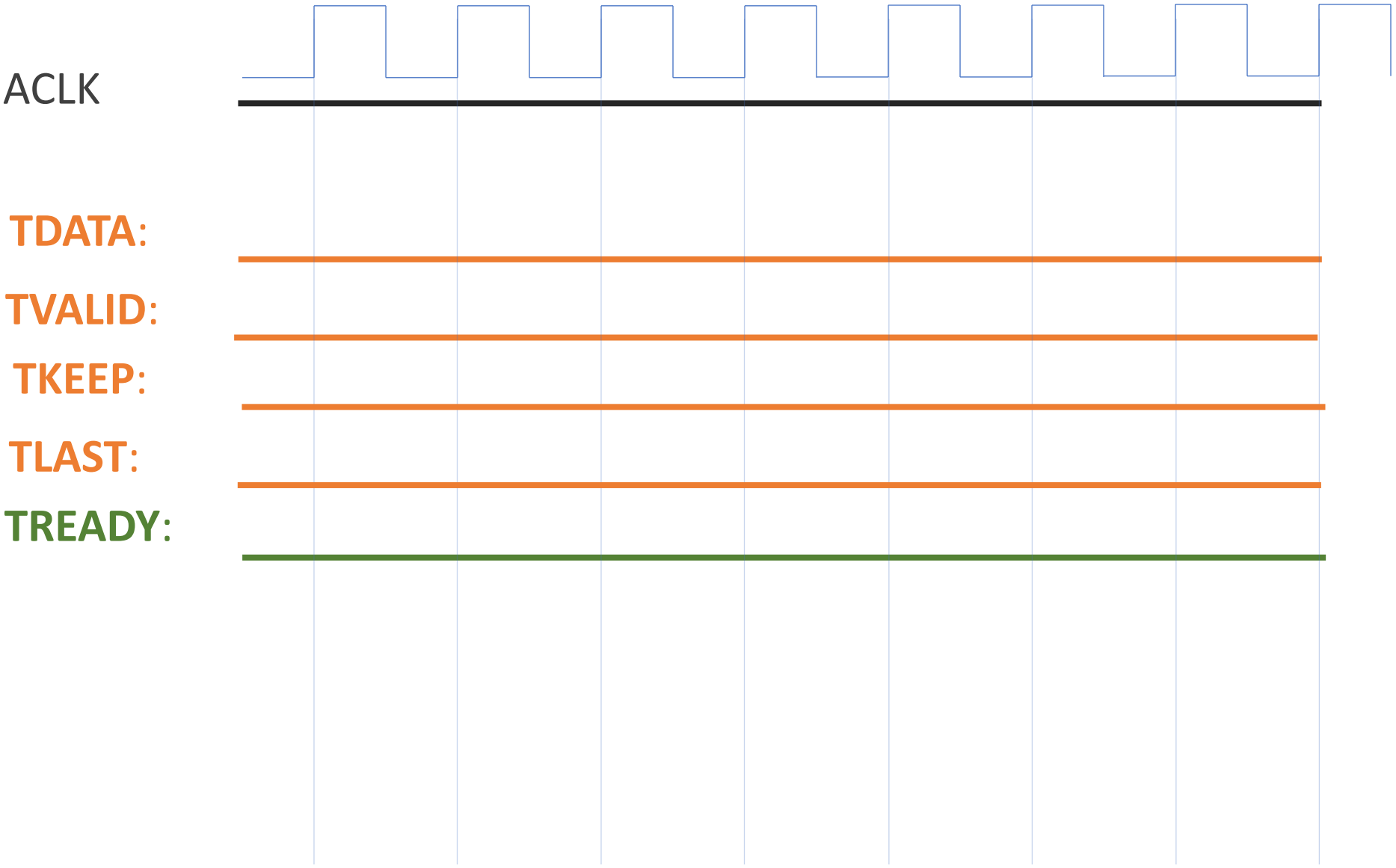
0xC
- **TKEEP** ==1100 -> Keep first 2 bytes (16-bit)

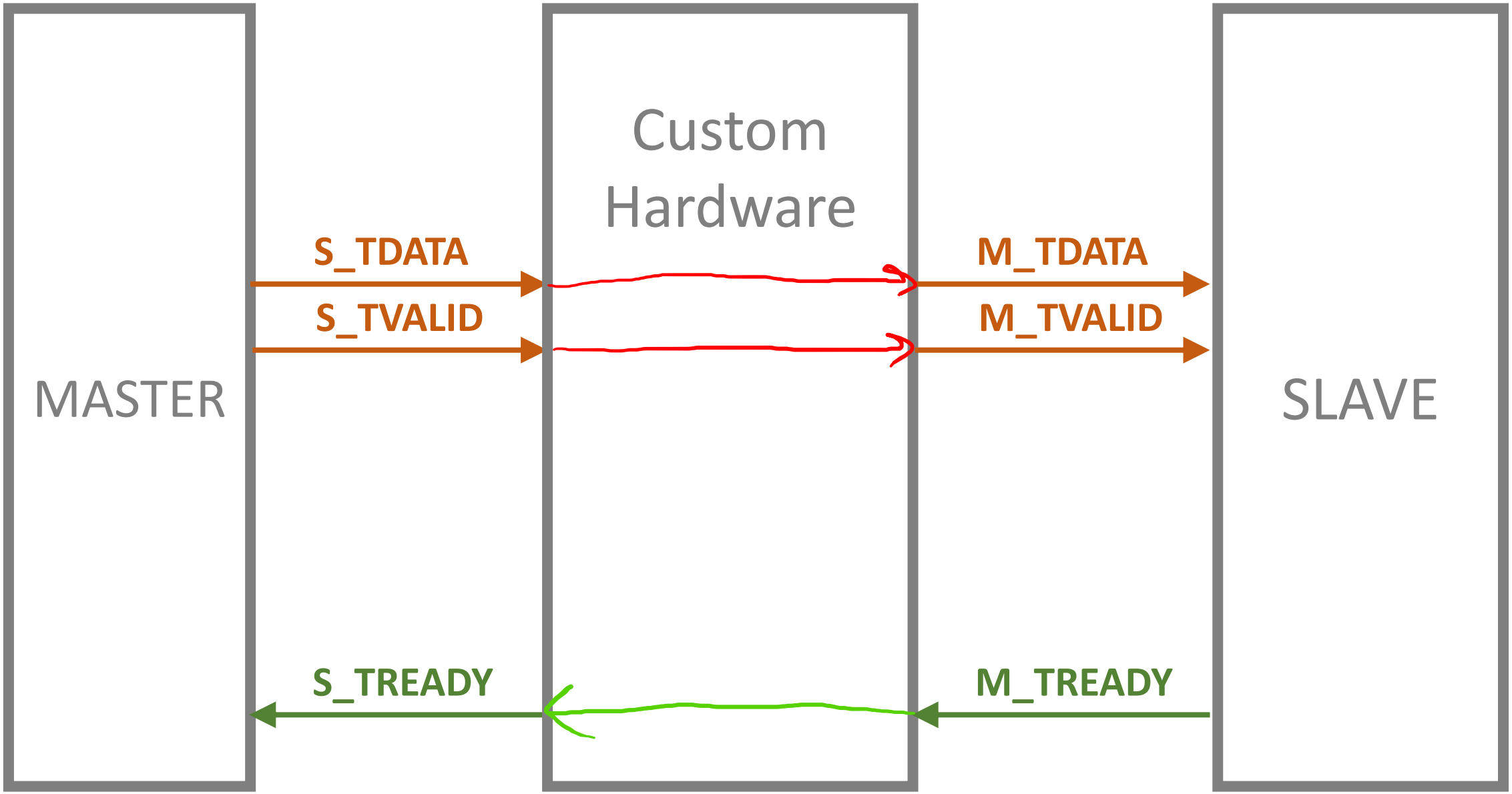
0x8
- **TKEEP** ==1000 -> Keep first byte (8-bit)

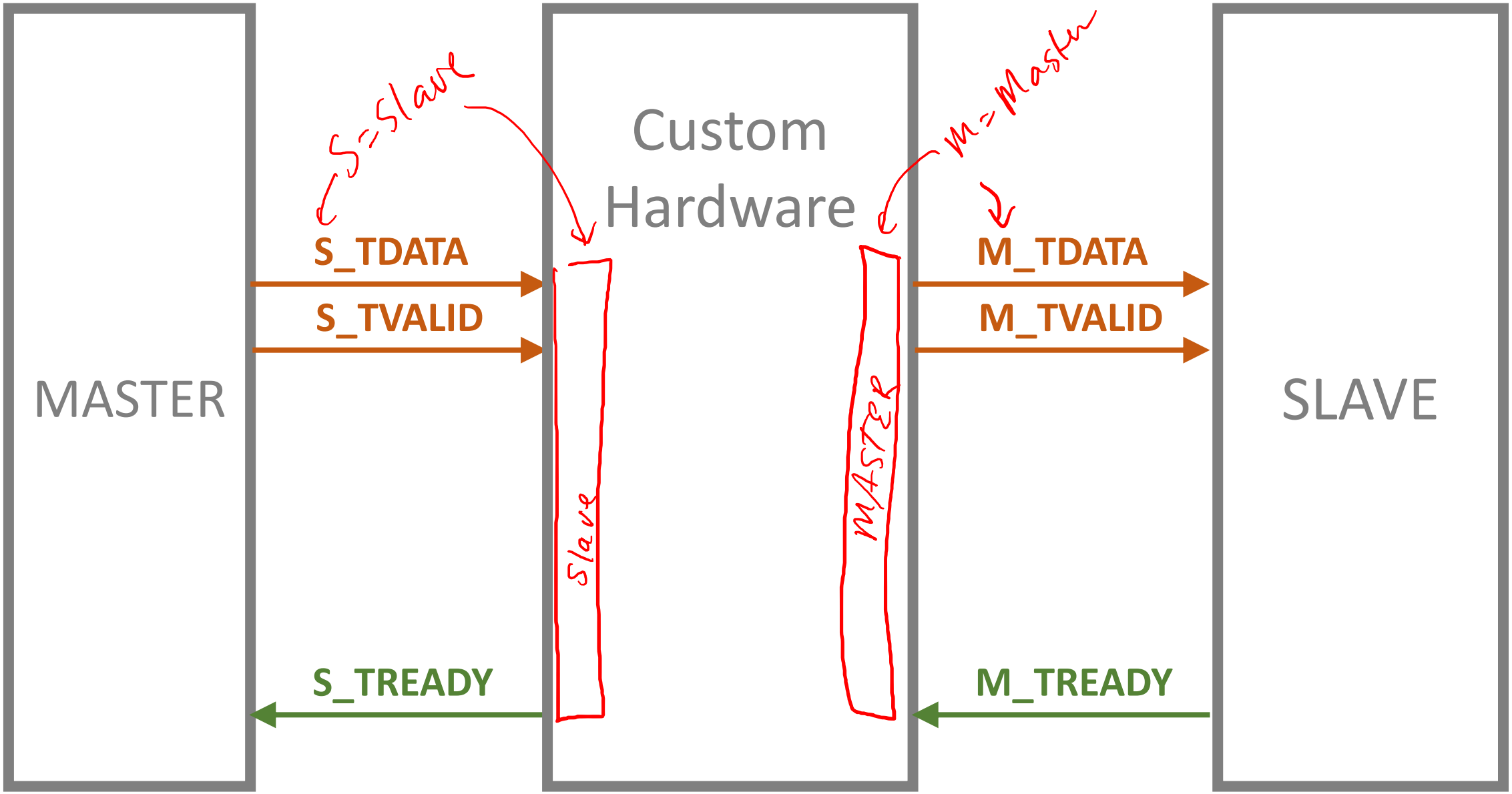
TLAST

- Special signal to indicate a group or “burst” of transmissions is complete.
- “Indicates the boundary of a packet”

Using TKEEP and TLAST




Custom
Hardware
MASTER
SLAVE
S_TDATA
S_TVALID
M_TDATA
M_TVALID
S_TREADY
M_TREADY

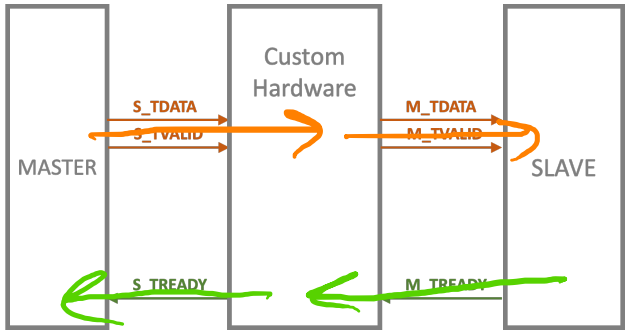

MASTER
S=Slave
Custom Hardware
m=Master
S_TDATA
S_TVALID
S_TREADY
Slave
MASTER
M_TDATA
M_TVALID
M_TREADY
SLAVE

Let's build a custom block that does nothing!

```
module custom_hw (
        input          ACLK,
        input          ARESET,
        input [31:0]   S_TDATA,
        input          S_TVALID,
        output         S_TREADY,
        output [31:0]  M_TDATA,
        output         M_TVALID,
        input          M_TREADY
);
```

slave

MASTER

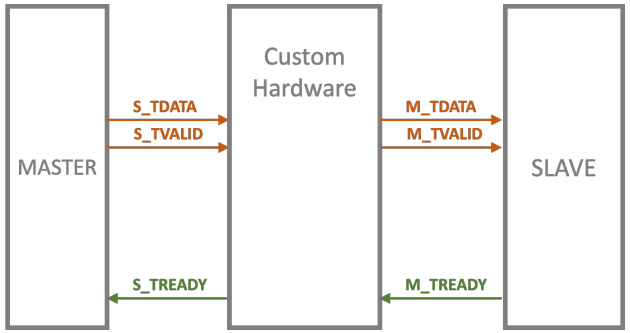


write me!

```
assign M_TDATA = S_TDATA;
assign M_TREADY = S_TREADY;
assign S_TREADY = M_TREADY;

endmodule
```

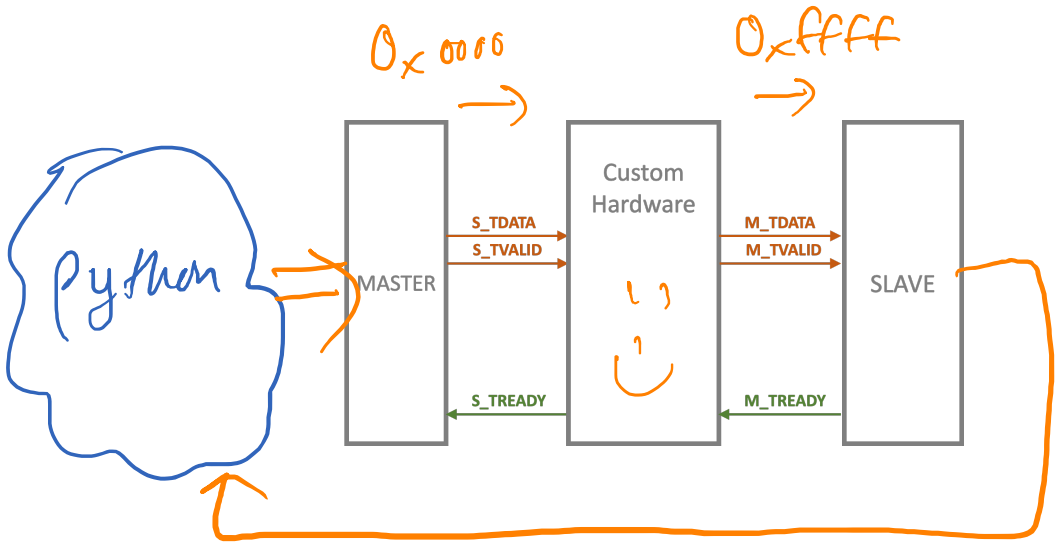
Let’s build a custom block that does nothing!

```
module custom_hw (
        input          ACLK,
        input          ARESET,
        input [31:0]   S_TDATA,
        input          S_TVALID,
        output         S_TREADY,
        output [31:0]  M_TDATA,
        output         M_TVALID,
        input          M_TREADY
);

assign M_TDATA = S_TDATA;
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;

endmodule
```



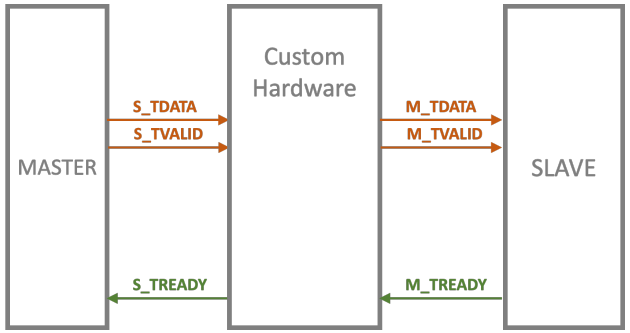
How would I flip all the bits of TDATA?

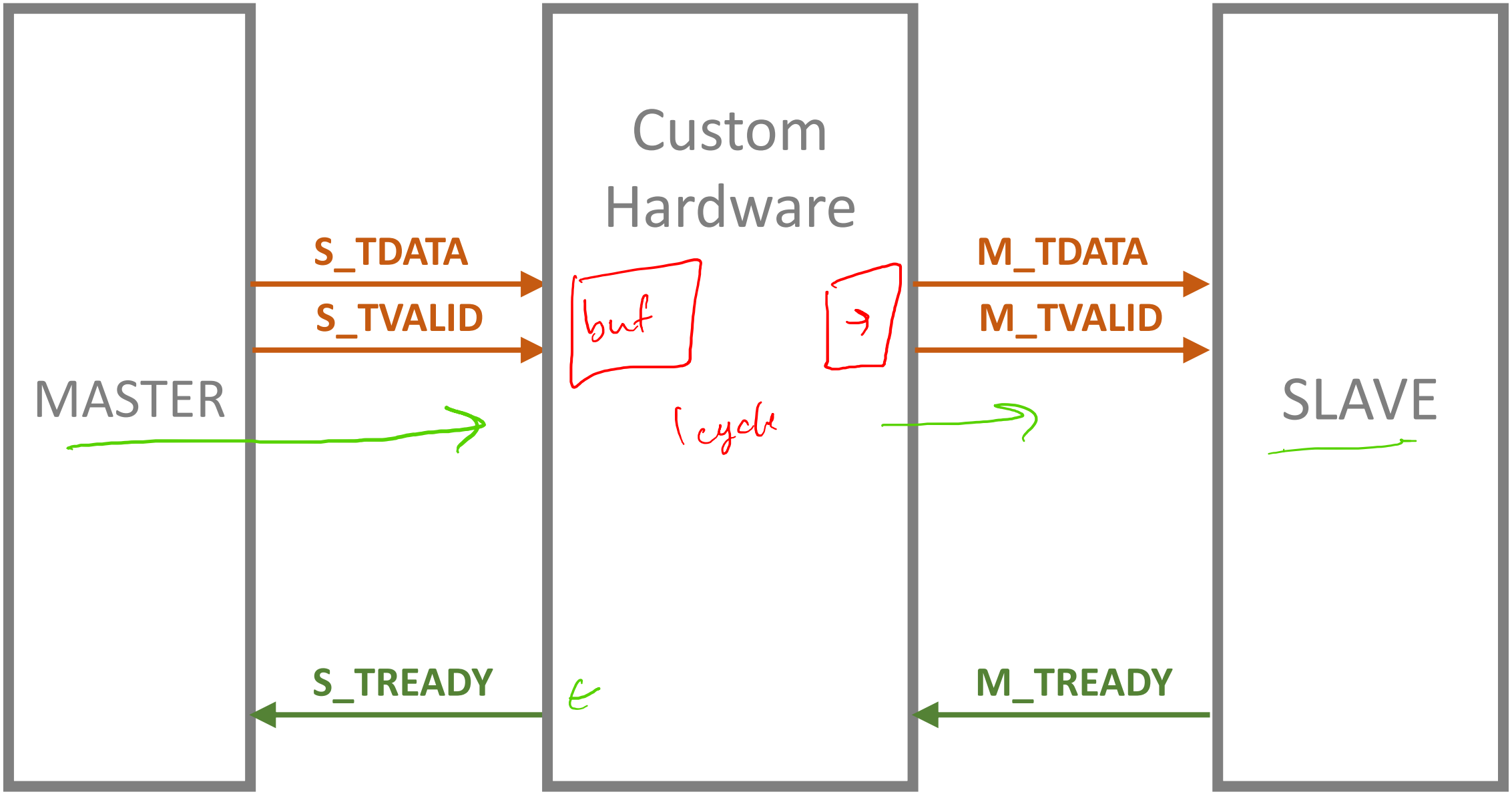
```
module custom_hw (
        input          ACLK,
        input          ARESET,
        input [31:0]   S_TDATA,
        input          S_TVALID,
        output         S_TREADY,
        output [31:0]  M_TDATA,
        output         M_TVALID,
        input          M_TREADY
);

assign M_TDATA = ~(S_TDATA);
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;

endmodule
```



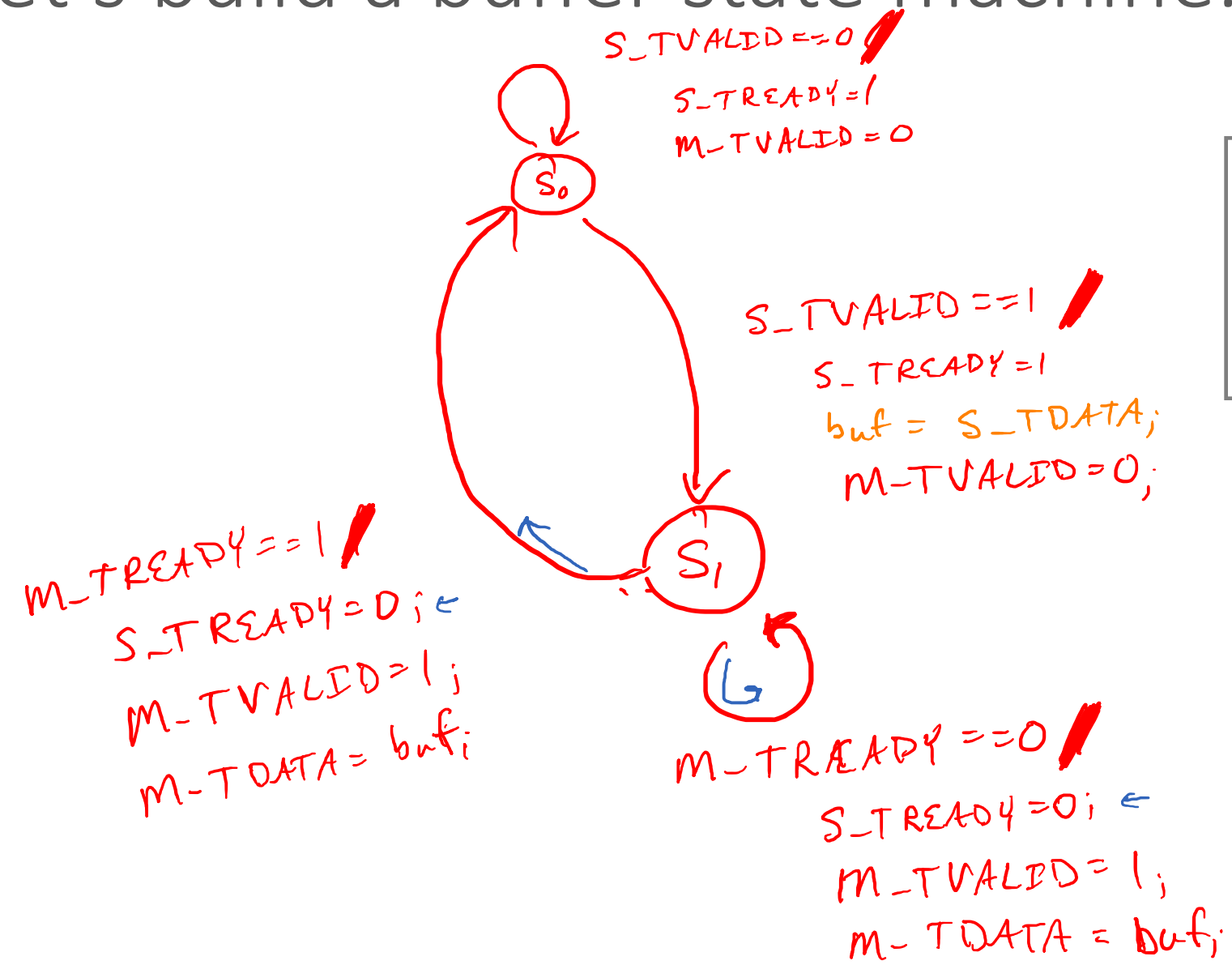
How would I flip all the bits of TDATA?

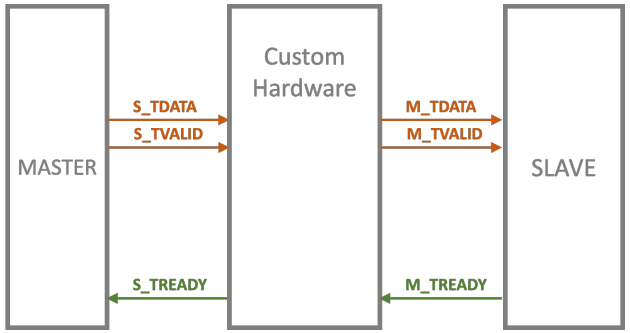
```
module custom_hw (
        input          ACLK,
        input          ARESET,
        input [31:0]   S_TDATA,
        input          S_TVALID,
        output         S_TREADY,
        output [31:0]  M_TDATA,
        output         M_TVALID,
        input          M_TREADY
);

assign M_TDATA = ~S_TDATA;
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;

endmodule
```




MASTER
S_TDATA
S_TVALID
S_TREADY
Custom Hardware
buf
1 cycle
M_TDATA
M_TVALID
M_TREADY
SLAVE

Let’s build a buffer state machine.





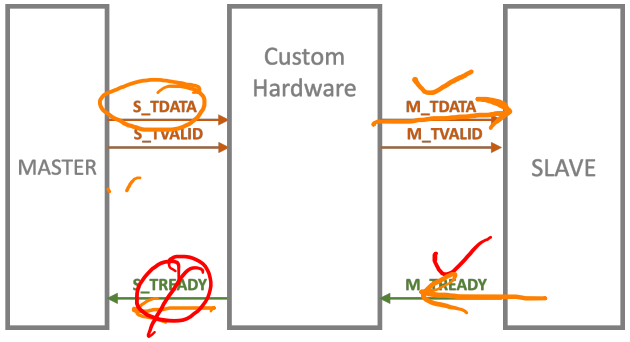
Let's build a buffer state machine.

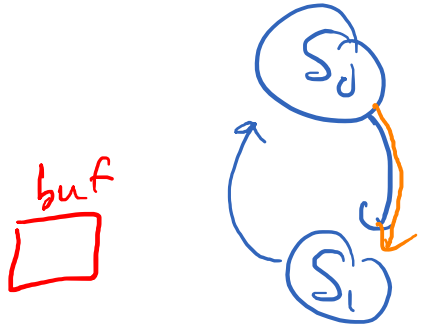
```
module custom_hw_buf (
        input           ACLK,
        input           ARESET,
        input [31:0]    S_TDATA,
        input           S_TVALID,
        output          S_TREADY,
        output [31:0]   M_TDATA,
        output          M_TVALID,
        input           M_TREADY
);
```

```
reg [31:0] buf, next buf;
enum { S0, S1 } state, nextState;
```

```
always_ff (@ posedge ACLK) begin
    if (ARESET) begin
        state <= S0;
        buf <= 32'h0;
    end else begin
        state <= nextState;
        buf <= next buf;
    end
end
```

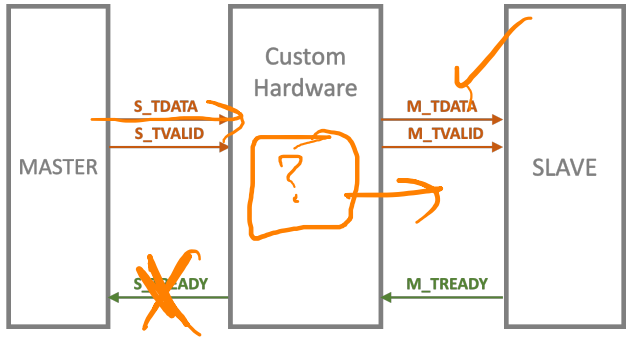
```
always_comb begin
    M_TDATA = buf
    M_TVALID = 1'h0;
    S_TREADY = 1'h1;
    next State = state;
    next buf = buf;
    case (state)
        S0: if (S_TVALID) begin
                next_buf = S_TDATA;
                next State = S1;
            end
        S1: S_TREADY = 0;
            M_TVALID = 1;
            if (M_TREADY)
                next State = S0
```






```
endmodule
```

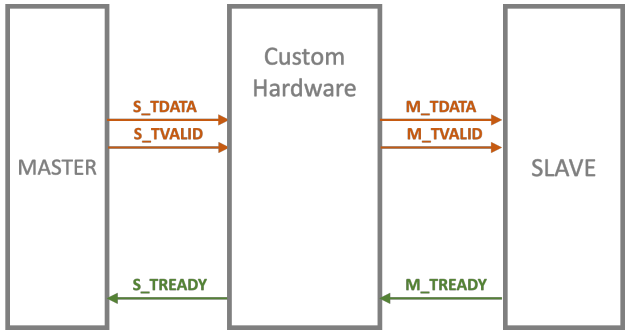
Let's build a buffer state machine.




```
module custom_hw_buf (
        input           ACLK,
        input           ARESET,
        input [31:0]    S_TDATA,
        input           S_TVALID,
        output          S_TREADY,
        output [31:0]   M_TDATA,
        output          M_TVALID,
        input           M_TREADY
);

enum {S0, S1} state, nextState;
reg [31:0] nextVal;

always_ff @(posedge ACLK) begin
    if (ARESET)begin
        state <= S0;
        M_TDATA <= 32'h0
    end else begin
        state <= nextState;
        M_TDATA <= nextVal;
    end
end
```

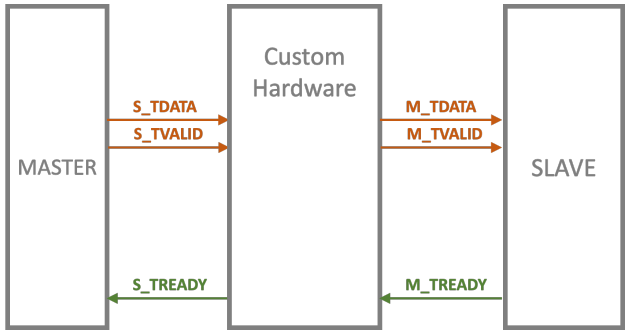
```
always_comb begin
    S_TREADY = 'h1;
    M_TVALID = 'h0;
    nextState = state;
    nextVal = M_TDATA;
    case(state)
        S0:  begin
            if (S_TVALID) begin
                nextState = S1;
                nextVal = S_TDATA;
            end
        end
        S1: begin
            S_TREADY = 'h0;
            M_TVALID = 'h1;
            if (M_TREADY) begin
                nextState = S0;
        end
    endcase
end

endmodule
```

Let’s build popcount.



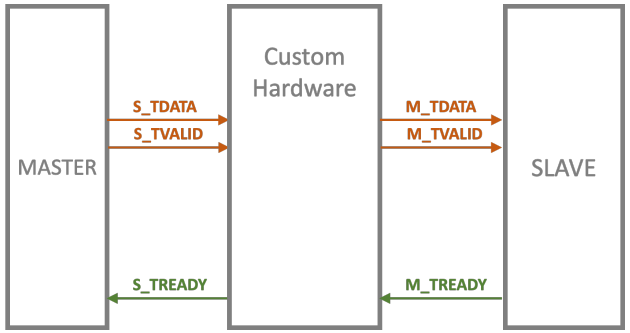
Let’s build popcount.

```
module custom_hw_popcount (
        input           ACLK,
        input           ARESET,
        input [31:0]    S_TDATA,
        input           S_TVALID,
        output          S_TREADY,
        output [31:0]   M_TDATA,
        output          M_TVALID,
        input           M_TREADY
);



endmodule
```



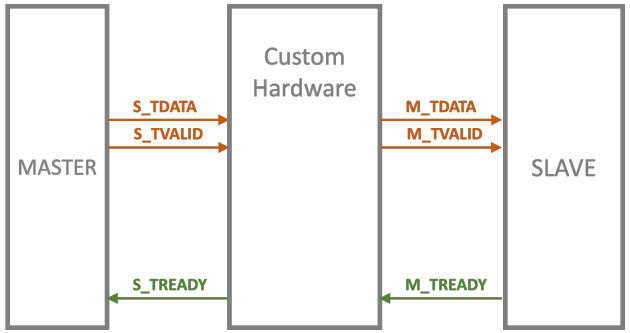
Let’s build popcount.

```
module custom_hw_popcount (
        input           ACLK,
        input           ARESET,
        input [31:0]    S_TDATA,
        input           S_TVALID,
        output          S_TREADY,
        output [31:0]   M_TDATA,
        output          M_TVALID,
        input           M_TREADY
);

assign M_TDATA = S_TDATA[0] + S_TDATA[1] + S_TDATA[2] + … + S_TDATA[30] + S_TDATA[31];
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;


endmodule
```



Let’s build popcount.

```
module custom_hw_popcount (
        input          ACLK,
        input          ARESET,
        input [31:0]   S_TDATA,
        input          S_TVALID,
        output         S_TREADY,
        output [31:0]  M_TDATA,
        output         M_TVALID,
        input          M_TREADY
);

integer i;
reg [31:0] tmp;

always_comb begin
    tmp = 32’h0;
    for (i = 0; i < 32; ++i) tmp += S_TDATA[i];
end

assign M_TDATA = tmp;
assign M_TVALID = S_TVALID;
assign S_TREADY = M_TREADY;

endmodule
```



Next Time

- Memory-Mapped I/O
- Memory-Mapped Buses

References

- Zynq Book, Chapter 19 “AXI Interfacing”
- Practical Introduction to Hardware/Software Codesign
  - Chapter 10
- AMBA AXI Protocol v1.0
  - http://mazsola.iit.uni-miskolc.hu/~drdani/docs_arm/AMBAaxi.pdf
- https://lauri.võsandi.com/hdl/zynq/axi-stream.html

05: Bus Interfaces

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University